



www.tresdefebrero.gov.ar/tecno3f

#ProgramáTuFuturo





Municipalidad de
Tres de Febrero



Programá
tu futuro



PYTHON INTERMEDIO

¡Les damos la bienvenida!





Municipalidad de
Tres de Febrero



Programá
tu futuro



PROGRAMACIÓN ORIENTADA A OBJETOS I

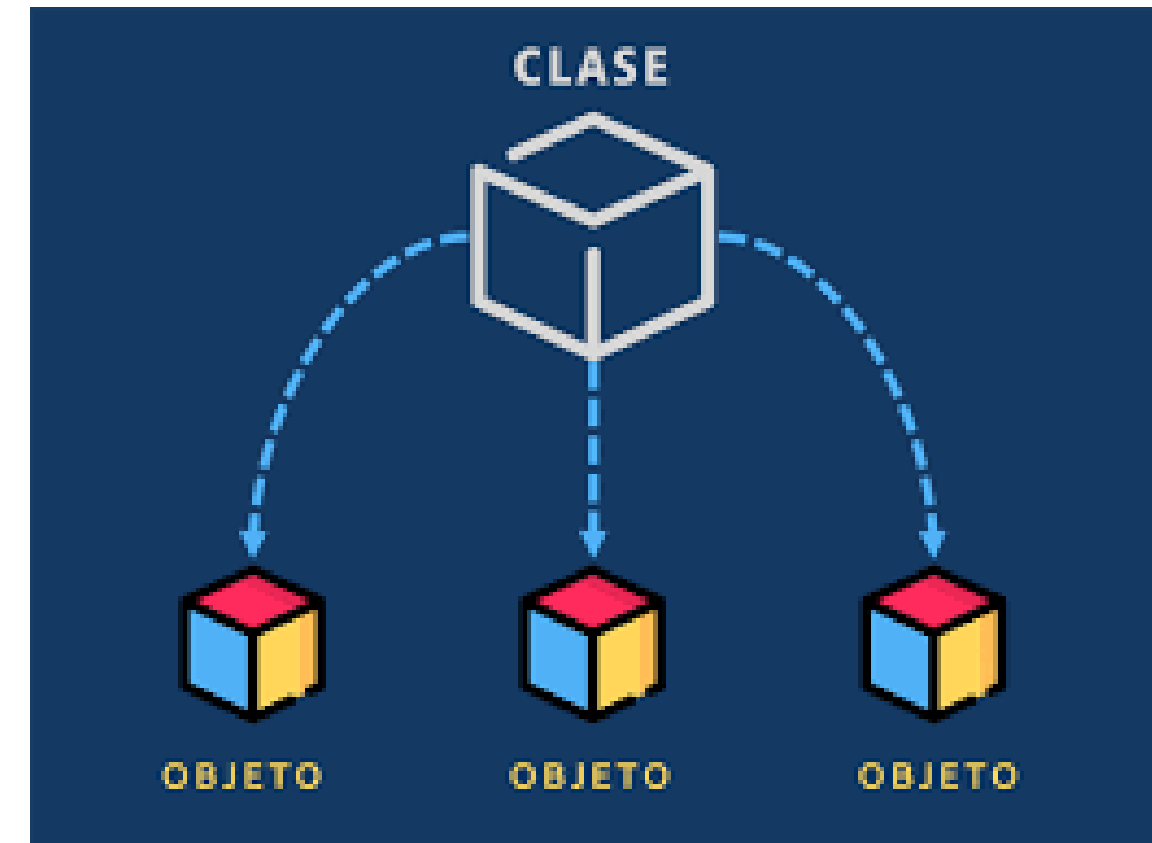
CLASE 4



¿QUÉ ES?

Paradigma Orientado a Objetos (POO): Modelo de programación que organiza el código en unidades llamadas objetos, combinando datos y acciones.

- Clase (El Molde / Plantilla): Define la estructura:
- Atributos: Características o estado (datos).
- Métodos: Comportamiento o acciones (funciones).
- Objeto (La Instancia Real): La entidad concreta creada en memoria a partir de una clase.



Ventajas

Reutilización de código: Creás una clase una sola vez y podés instanciarla o extenderla en múltiples partes del proyecto sin duplicar código.

Mantenimiento del código: Si cambian las reglas de negocio de una entidad (por ejemplo, cómo se calcula un descuento), solo editás esa clase y el resto del sistema sigue funcionando.

Modularidad: El sistema se divide en piezas autónomas y desacopladas. Si falla un objeto, no se rompe toda la aplicación.

Escalabilidad y Extensibilidad: Permite sumar nuevas funciones o tipos de datos agregando nuevas clases sin tener que reescribir el código base existente

Anatomía de una Entidad en POO

Atributos (Estado / Datos):

- Son las variables internas asociadas al objeto.
- Responden a: ¿Cómo es? / ¿Qué datos tiene?
- Ejemplo auto: **marca, color, velocidad, encendido.**

Métodos (Comportamiento / Acciones):

- Son las funciones definidas dentro de la clase que manipulan esos atributos.
- Responden a: ¿Qué puede hacer? / ¿Cómo cambia?
- Ejemplo auto: **encender(), acelerar(), frenar().**



Pregunta

¿Aparte de *Python*, qué lenguaje conocen o les parece que también puede usar POO?

Desbloquen el microfono o escriban por el chat.



LOS 4 PILARES



Programá
tu futuro



Municipalidad de
Tres de Febrero

Encapsulamiento

Agrupar los datos (atributos) y los métodos que operan sobre ellos en una misma unidad, controlando y restringiendo el acceso directo a los detalles internos del objeto para proteger su integridad.

Puntos clave y Ventajas

- **Protección del estado:** Evita que el código exterior modifique variables críticas de forma indebida o con valores inválidos (por ejemplo, asignar una velocidad negativa).
- **Interfaz controlada:** Los cambios de estado se hacen exclusivamente mediante métodos autorizados (getters, setters o métodos de acción) que validan las reglas de negocio.
- **Mantenibilidad:** Si cambia la forma de almacenar un dato por dentro, la forma en que el resto del programa interactúa con el objeto no se rompe.

Herencia

Mecanismo jerárquico que permite definir una nueva clase (clase hija o subclase) a partir de una existente (clase padre o superclase), heredando sus atributos y métodos, y permitiendo añadir nuevos o modificar los existentes.

Puntos clave / Ventajas

- **Principio "Es un" (Is-a):** Modela relaciones naturales del dominio (un Camion es un Vehiculo, un Administrador es un Usuario).
- **Reutilización y DRY (Don't Repeat Yourself):** Todo el código común vive en la clase padre; no se duplican lógica ni atributos entre clases similares.
- **Especialización:** La clase hija aprovecha lo general y solo implementa lo específico de su comportamiento.

Polimorfismo

Del griego "muchas formas". Es la capacidad de distintos objetos de responder a una misma llamada a un método, implementando cada uno su propio comportamiento específico según su clase.

Puntos clave y Ventajas

- **Intercambiabilidad:** Una función puede trabajar con cualquier objeto sin importar su clase exacta, siempre que implemente el método esperado.
- **Eliminación de condicionales innecesarios:** Evita llenar el código de comprobaciones tipo `if tipo == 'Perro': ... elif tipo == 'Gato': ...`
- **Duck Typing (Polimorfismo en Python):** No requiere jerarquías rígidas de herencia previa; basta con que el objeto posea el método solicitado.

Abstracción

Consiste en modelar únicamente los aspectos esenciales de un objeto para el problema específico que queremos resolver, ignorando los detalles complejos o irrelevantes del mundo real.

Puntos clave y Ventajas

- **Foco en el dominio:** Un objeto Auto en un sistema de peajes solo necesita patente y categoría; en un videojuego de carreras necesita velocidad_maxima y traccion. El contexto define qué abstraer.
- **Reducción de complejidad:** Permite interactuar con sistemas sofisticados mediante una interfaz simple (solo llamás a .acelerar()) sin necesidad de entender la inyección electrónica interna).
- **Mantenibilidad:** Aislar la interfaz del funcionamiento interno permite cambiar la lógica profunda sin romper cómo lo usan los demás componentes.

TEST



Programá
tu futuro



Municipalidad de
Tres de Febrero

¿A que Pilar se hace referencia?

Al diseñar un sistema para una veterinaria, modelamos la entidad **Mascota** registrando únicamente su nombre, especie y fecha_nacimiento.

Se descartan deliberadamente detalles irrelevantes como el color exacto de sus ojos , para mantener el sistema simple y manejable.

Polimorfismo

Abstracción

Encapsulamiento

Herencia

¿A que Pilar se hace referencia?

Al diseñar un sistema para una veterinaria, modelamos la entidad **Mascota** registrando únicamente su nombre, especie y fecha_nacimiento.

Se descartan deliberadamente detalles irrelevantes como el color exacto de sus ojos , para mantener el sistema simple y manejable.

Polimorfismo

Abstracción

Encapsulamiento

Herencia

¿A que Pilar se hace referencia?

Imagina una clase **Coche** con el atributo **_velocidad**.

Para evitar que alguien altere la velocidad directamente con valores erróneos o negativos desde el exterior, la velocidad solo puede modificarse invocando al método **acelerar()** o **frenar()**.

Polimorfismo

Abstracción

Encapsulamiento

Herencia

¿A que Pilar se hace referencia?

Imagina una clase **Coche** con el atributo **_velocidad**.

Para evitar que alguien altere la velocidad directamente con valores erróneos o negativos desde el exterior, la velocidad solo puede modificarse invocando al método **acelerar()** o **frenar()**.

Polimorfismo

Abstracción

Encapsulamiento

Herencia

¿A que Pilar se hace referencia?

Imagina una función **hacer_sonar_alarma(vehiculo)** que recibe cualquier tipo de vehículo (Auto, Moto o Camion) e invoca su método **.activar_alarma()**.

Cada vehículo reacciona a su manera: el auto hace sonar una sirena, la moto emite beeps y el camión enciende balizas, sin necesidad de modificar la función ni escribir condicionales (if/elif).

Polimorfismo

Abstracción

Encapsulamiento

Herencia

¿A que Pilar se hace referencia?

Imagina una función **hacer_sonar_alarma(vehiculo)** que recibe cualquier tipo de vehículo (Auto, Moto o Camion) e invoca su método **.activar_alarma()**.

Cada vehículo reacciona a su manera: el auto hace sonar una sirena, la moto emite beeps y el camión enciende balizas, sin necesidad de modificar la función ni escribir condicionales (if/elif).

Polimorfismo

Abstracción

Encapsulamiento

Herencia

¿A que Pilar se hace referencia?

Imagina una clase general llamada **Vehiculo** con atributos como marca, modelo y el método arrancar().

Luego creamos las clases Auto, Moto y Camion, las cuales adquieren automáticamente todos los atributos y métodos de **Vehiculo** sin tener que volver a escribirlos desde cero, sumando únicamente sus datos propios.

Polimorfismo

Abstracción

Encapsulamiento

Herencia

¿A que Pilar se hace referencia?

Imagina una clase general llamada **Vehiculo** con atributos como marca, modelo y el método arrancar().

Luego creamos las clases Auto, Moto y Camion, las cuales adquieren automáticamente todos los atributos y métodos de **Vehiculo** sin tener que volver a escribirlos desde cero, sumando únicamente sus datos propios.

Polimorfismo

Abstracción

Encapsulamiento

Herencia

CREANDO NUESTRO PRIMER OBJETO



Programá
tu futuro



Municipalidad de
Tres de Febrero

Creando la Clase y el Constructor

```
1 class Coche:  
2     pass
```

Con la palabra reservada **class** , creamos la clase y le asignamos un nombre

Luego procedemos a crear su
Constructor

```
class Coche:  
    def __init__(self, marca, modelo, color):  
        self.marca = marca  
        self.modelo = modelo  
        self.color = color
```

¿Que es un Constructor?

Es un método que se encarga de inicializar un objeto de la clase, es decir, de asignarle valores a sus **atributos**.

En este caso

- `self` es una referencia al objeto que se está creando.
- `marca`, `modelo` y `color` son los parámetros del método constructor. Estos parámetros se utilizan para inicializar los atributos del objeto.

Hagamos una prueba **Instanciamos** la clase **Coche** y creemos dos objetos

```
coche = Coche("Fiat", "Duna", "Rojo")  
coche2 = Coche("Renault", 12, "Azul Metalizado")
```

Leyendo Valores

Bien ahora que ya instanciamos la clase Coche, y creamos dos objetos, podemos acceder y ver sus valores

```
print(coche.marca)
print(coche.modelo)
print(coche.color)
print("\n")
print(coche2.marca)
print(coche2.modelo)
print(coche2.color)
```

Como se ve,
pudimos crear 2
objetos diferentes a
partir de la misma
clase

```
S-2024/Clase 4/01
Fiat
Duna
Rojo

Renault
12
Azul Metalizado
```

Agregando acciones

Venimos excelente, ahora agreguemos algunos metodos.

```
class Coche:
    def __init__(self, marca, modelo, color):
        self.marca = marca
        self.modelo = modelo
        self.color = color
        self.encendido = False

    def encender(self):
        self.encendido = True
        print(f"El coche {self.marca} {self.modelo} está encendido.")

    def apagar(self):
        self.encendido = False
        print(f"El coche {self.marca} {self.modelo} está apagado.")
```

Atributo de estado inicial por defecto (no hace falta pedirlo por parámetro en el constructor)

Bien, como primera medida, agregaremos un nuevo atributo llamado encendido y creamos los metodos de encender y apagar

¿Qué hacen estas acciones?

En este caso las acciones modificarán el atributo **encendido** cambiando su estado.

```
def encender(self):  
    self.encendido = True  
    print(f"El coche {self.marca} {self.modelo} está encendido.")
```

Prueba

```
print(coche.encendido) #estado del coche antes de la accion  
coche.encender() #se realiza la accion  
print(coche.encendido) #estado del coche luego de la accion  
print("\n")  
print(coche2.encendido) #estado del coche2
```

Resultado

```
False  
El coche Fiat Duna está encendido.  
True
```

```
False  
garo-pc@garopc-MS-7C52: ~/Documentos /
```

¡MUCHAS GRACIAS!



Programá
tu futuro



Municipalidad de
Tres de Febrero